Laporan Tugas Mandiri Modul 10

API dengan MongoDB — Portofolio Mahasiswa

[Nama Lengkap] NIM 434241021
Repository: https://github.com/vyerru/TUGAS1_434241021

28 Oktober 2026

Contents

C.1 — Rancangan Dokumen Portofolio **2**

C.2 — Potongan Kode Penting **4**

C.3 — Bukti Pengujian **9**

C.4 — Jawaban Pertanyaan Analisis **13**

C.1 — Rancangan Dokumen Portofolio

1.1 Struktur koleksi `student_portfolios`

Portofolio mahasiswa disimpan sebagai **satu dokumen per mahasiswa** di koleksi `student_portfolios` (MongoDB), sedangkan data induknya (`students`) tetap di PostgreSQL. Rujukan keduanya adalah `student_id` — **bukan** foreign key, karena MongoDB dan PostgreSQL tidak bisa saling menjaga integritas; keberadaan mahasiswa selalu diperiksa ke PostgreSQL di lapisan service (lihat potongan `student0wned` pada bagian C.2).

Field	Tipe	Keterangan
`_id`	ObjectId	Identitas dokumen; **dipertahankan** saat PUT ulang
`student_id`	int	Rujukan logis ke `students.id` (PostgreSQL)
`headline`	string	Judul singkat portofolio (3–120 karakter)
`bio`	string	Deskripsi diri (maks. 2000 karakter)
`projects[]`	array dokumen **embedded**	Maks. 20 proyek; tiap proyek punya `_id` sendiri untuk endpoint DELETE
`projects[].technologies[]`	array string	Dinormalisasi: huruf kecil, tanpa spasi tepi, tanpa duplikat
`projects[].link`	string, opsional	URL repository; field hilang (`omitempty`) bila kosong
`certificates[]`	array dokumen **embedded**	Tanpa `_id` — tidak ada endpoint yang menghapus satu sertifikat sendirian
`created_at`	date	Ditentukan saat dokumen pertama dibuat; **tidak berubah** saat PUT ulang
`updated_at`	date	Diperbarui setiap perubahan

Projects dan sertifikat **disematkan** (embedding) dalam dokumen induk karena: selalu dibaca bersama portofolio (tidak ada endpoint "baca satu proyek sendirian"), jumlahnya terbatas (20 proyek / 50 sertifikat — jauh di bawah batas dokumen 16 MB), dan tidak diakses lintas dokumen.

1.2 Contoh satu dokumen lengkap

Dokumen asli milik mahasiswa `student_id: 71` (diambil langsung dari MongoDB dengan `mongosh`, `_id` konversi ObjectId ke heksadesimal):

```
{
  "_id": "6ae1b8c498574287607c9b0c",
```

```
  "student_id": 71,
  "headline": "Fullstack Learner",
  "bio": "Suka web dan data",
  "projects": [
    {
      "_id": "6ae1ba9998574287607c9b28",
      "title": "Toko Online",
      "technologies": ["python", "django", "docker"],
      "link": "https://github.com/rev002/toko",
      "year": 2025
    },
    {
      "_id": "6ae1ba9998574287607c9b29",
      "title": "Scraper Berita",
      "technologies": ["go", "docker"],
      "year": 2025
    },
    {
      "_id": "6ae1ba9998574287607c9b2a",
      "title": "Analitik Penjualan",
      "technologies": ["python", "postgresql"],
      "year": 2024
    }
  ],
  "certificates": [
    {
      "name": "Database Design Fundamentals",
      "issuer": "Oracle Academy",
      "date": "2024-11-20T00:00:00Z"
    }
  ],
  "created_at": "2026-10-28T08:56:36.104Z",
  "updated_at": "2026-10-28T09:04:25.063Z"
}
```


Pada dokumen ini `created_at` ≠ `updated_at` karena portofolionya diisi ulang (sepertifikat ditambahkan) — identitas dokumen (`_id` dan `created_at`) tetap, sesuai desain.

1.3 Keputusan index

Hanya **satu** index yang dibuat: `student_portfolio_unique` pada `{student_id: 1}` dengan opsi `unique`. Dua alasannya:

1. **Menjaga aturan bisnis di level database** — "satu mahasiswa tepat satu portofolio" tidak bisa dilanggar oleh dua request PUT serentak, karena index unik menolak dokumen kedua dengan `student_id` yang sama.
2. **Melayani seluruh pola akses** — semua endpoint mengakses portofolio lewat `student_id` (GET, PUT, POST/DELETE proyek), sehingga index ini dipakai di mana-mana.

Index lain sengaja **tidak** dibuat: endpoint statistik teknologi membaca *seluruh* koleksi

(aggregation tanpa `$match`), sehingga index selective apa pun tidak akan dipakai di sana dan hanya menambah biaya tulis. Bukti `explain` ada di bagian C.3.

C.2 — Potongan Kode Penting

Kode lengkap ada di `MODUL_10/app/repository/portfolio_repository.go` dan `MODUL_10/app/service/portfolio_service.go` (commit C.1–C.2.4). Berikut bagian-bagian paling menentukan (dipangkas dari komentar panjang).

2.1 Pemeriksaan lintas database — `studentOwned`

Tidak ada foreign key lintas database; keberadaan mahasiswa hanya bisa ditanyakan ke PostgreSQL, lalu otorisasi memakai aturan yang sama dengan endpoint student (pemilik, atau permission `student:read:any` / `student:update:any`):

```go
func (s *PortfolioService) studentOwned(
    ctx context.Context, c *fiber.Ctx, anyPermission string,
) (model.Student, error) {
    current, ok := helper.CurrentUser(c)
    if !ok {
        return model.Student{}, helper.Unauthorized("belum terautentikasi")
    }
    studentID, valid := helper.ParamID(c)
    if !valid {
        return model.Student{}, helper.BadRequest(
            "id mahasiswa harus berupa angka positif")
    }

    student, err := s.students.FindByID(ctx, studentID) // PostgreSQL
    if err != nil {
        return model.Student{}, translatePortfolioError(err, "mahasiswa")
    }
    if !CanAccessStudent(current, student.OwnerID, s.perms, anyPermission) {
        return model.Student{}, helper.Forbidden(
            "tidak berhak mengakses portofolio mahasiswa milik orang lain")
    }
    return student, nil
}
```

Karena pemeriksaan ini dijalankan **sebelum** menyentuh MongoDB, GET portofolio milik mahasiswa yang tidak ada di PostgreSQL mengembalikan 404 “mahasiswa tidak ditemukan” — bukan 404 portofolio (bukti C2 di C.3).

2.2 Index unik — `EnsureIndexes`

```go
func (r *portfolioMongoRepository) EnsureIndexes(ctx context.Context) error {
    _, err := r.coll.Indexes().CreateMany(ctx, []mongo.IndexModel{
        {
            Keys:    bson.D{{Key: "student_id", Value: 1}},
```

```
            Options: options.Index().SetUnique(true).
                SetName("student_portfolio_unique"),
        },
    })
    if err != nil {
        return fmt.Errorf("membuat index student_portfolios: %w", err)
    }
    return nil
}
```

2.3 PUT semantik upsert — Replace

Portofolio dibuat atau diganti seluruhnya. Kuncinya: baca dulu dokumen lama untuk **mempertahankan _id dan created_at**, lalu FindOneAndReplace dengan upsert. Bila dua PUT serentak sama-sama mendapati portofolio belum ada, index unik menolak yang kedua (duplicate key) — percobaan diulang:

```
func (r *portfolioMongoRepository) Replace(
    ctx context.Context, studentID int, p model.Portfolio,
) (model.Portfolio, error) {
    for attempt := 0; attempt < 3; attempt++ {
        doc := portfolioDocument{ /* ... isi dari p, CreatedAt: now() ... */ }

        var existing portfolioDocument
        err := r.coll.FindOne(ctx, bson.M{"student_id": studentID}).Decode(&existing
        switch {
        case err == nil:
            doc.ID = existing.ID          // identitas tetap
            doc.CreatedAt = existing.CreatedAt
        case errors.Is(err, mongo.ErrNoDocuments):
            doc.ID = bson.NewObjectID()
        default:
            return model.Portfolio{}, fmt.Errorf("memeriksa portofolio: %w", err)
        }

        opts := options.FindOneAndReplace().
            SetUpsert(true).
            SetReturnDocument(options.After)

        err = r.coll.FindOneAndReplace(ctx,
            bson.M{"student_id": studentID}, doc, opts).Decode(&doc)
        if err == nil {
            return doc.toModel(), nil
        }
        if mongo.IsDuplicateKeyError(err) {
            continue // dibuat proses lain di sela pembacaan; ulangi
        }
        return model.Portfolio{}, fmt.Errorf("menyimpan portofolio: %w", err)
    }
```

```
    return model.Portfolio{},
        fmt.Errorf("menyimpan portofolio: terlalu banyak percobaan bersamaan")
}
```

2.4 Batas 20 proyek, atomik — `AddProject`

Pemeriksaan batas dan penambahan terjadi dalam **satu** operasi atomik: filter `projects.19 $exists false` berarti "array belum berisi 20 elemen", sehingga dua request serentak tidak mungkin sama-sama lolos saat sisa satu tempat — penambahan ke-21 ditolak langsung oleh database:

```
func (r *portfolioMongoRepository) AddProject(
    ctx context.Context, studentID int, p model.Project,
) (model.Portfolio, error) {
    limitField := "projects." + strconv.Itoa(MaxProjectsPerPortfolio-1) // projects.

    doc := projectDocument{
        ID: bson.NewObjectID(), Title: p.Title,
        Technologies: p.Technologies, Link: p.Link, Year: p.Year,
    }
    if doc.Technologies == nil {
        doc.Technologies = []string{}
    }

    var updated portfolioDocument
    err := r.coll.FindOneAndUpdate(ctx,
        bson.M{"student_id": studentID, limitField: bson.M{"$exists": false}},
        bson.M{
            "$push": bson.M{"projects": doc},
            "$set":  bson.M{"updated_at": now()},
        },
        options.FindOneAndUpdate().SetReturnDocument(options.After),
    ).Decode(&updated)
    if err == nil {
        return updated.toModel(), nil
    }
    if !errors.Is(err, mongo.ErrNoDocuments) {
        return model.Portfolio{}, fmt.Errorf("menambah proyek: %w", err)
    }

    count, err := r.coll.CountDocuments(ctx, bson.M{"student_id": studentID})
    if err != nil {
        return model.Portfolio{}, fmt.Errorf("memeriksa portofolio: %w", err)
    }
    if count == 0 {
        return model.Portfolio{}, ErrNotFound // portofolionya memang tidak ada
    }
    return model.Portfolio{}, ErrProjectLimit // ada, tapi penuh -> 409
}
```

2.5 Hapus proyek dengan `$pull` — `DeleteProject`

Keberadaan proyek diperiksa **di dalam filter** update, sehingga cek dan hapus menjadi satu operasi. Bila tidak ada yang cocok, `CountDocuments` membedakan penyebabnya (pola *whyNoMatch*): portofolio tidak ada → 404 portofolio, portofolio ada tapi proyeknya tidak → 404 proyek:

```
func (r *portfolioMongoRepository) DeleteProject(
    ctx context.Context, studentID int, projectID string,
) (model.Portfolio, error) {
    oid, err := bson.ObjectIDFromHex(projectID)
    if err != nil {
        return model.Portfolio{}, ErrInvalidID // -> 400
    }

    var updated portfolioDocument
    err = r.coll.FindOneAndUpdate(ctx,
        bson.M{"student_id": studentID, "projects._id": oid},
        bson.M{
            "$pull": bson.M{"projects": bson.M{"_id": oid}},
            "$set":  bson.M{"updated_at": now()},
        },
        options.FindOneAndUpdate().SetReturnDocument(options.After),
    ).Decode(&updated)
    if err == nil {
        return updated.toModel(), nil
    }
    if !errors.Is(err, mongo.ErrNoDocuments) {
        return model.Portfolio{}, fmt.Errorf("menghapus proyek: %w", err)
    }

    count, err := r.coll.CountDocuments(ctx, bson.M{"student_id": studentID})
    if count == 0 {
        return model.Portfolio{}, ErrNotFound
    }
    return model.Portfolio{}, ErrProjectNotFound
}
```

2.6 Statistik teknologi — `$unwind` ganda

`$unwind` pertama memecah array `projects` (satu portofolio berisi 3 proyek menjadi 3 dokumen), `$unwind` kedua memecah array `technologies` milik tiap proyek, lalu `$group` menghitung tiap pasangan (proyek, teknologi) secara terpisah. Diurutkan menurun dan dibatasi 10 teratas:

```
pipeline := mongo.Pipeline{
    {{Key: "$unwind", Value: "$projects"}},
    {{Key: "$unwind", Value: "$projects.technologies"}},
    {{Key: "$group", Value: bson.M{
        "_id":   "$projects.technologies",
```

```
            "total": bson.M{"$sum": 1},
        }}},
        {{Key: "$sort", Value: bson.D{
            {Key: "total", Value: -1},
            {Key: "_id", Value: 1},
        }}},
        {{Key: "$limit", Value: 10}},
    }
```

C.3 — Bukti Pengujian

3.1 Lingkungan dan data uji

Komponen	Nilai
Aplikasi	MODUL_10 (Go 1.26, Fiber v2, mongo-driver v2.9.2), port 3000
Database	PostgreSQL (data mahasiswa) + MongoDB 8.0.30 (portofolio)
User uji	`rev001` (id 68), `rev002` (69), `rev003` (70), `revadmin` (71, role admin)
Mahasiswa uji	S1 = id 70 (NIM 434241101, pemilik `rev001`), S2 = id 71 (NIM 434241102, pemilik `rev002`)
Log bukti lengkap	`MODUL_10/bukti_uji_tugas_mandiri.txt`

3.2 Ringkasan hasil uji

#	Uji	Harapan	Hasil
C1	GET portofolio S1 yang belum dibuat	404 portofolio	OKlulus
C2	GET portofolio mahasiswa 99999 (tidak ada di PostgreSQL)	404 mahasiswa	OKlulus
C3	`rev002` PUT portofolio milik `rev001`	403	OKlulus
C4	`rev003` (bukan pemilik) GET portofolio S1	403	OKlulus
C5	PUT dengan teknologi kacau `["Go","PostgreSQL","go","Docker"]`	ternormalisasi	OK`["go","postgresql","docke`
C6	GET ulang setelah PUT pertama	`created_at` = `updated_at`	OKlulus
C7	PUT ulang (ganti headline)	`created_at` tetap	OKlulus
C8	POST proyek, teknologi `["Go","Redis","go"]`	201, ternormalisasi	OK`["go","redis"]`
C9	NoSQL injection pada body PUT	400	OKlulus

#	Uji	Harapan	Hasil
C10	DELETE proyek dengan id bukan ObjectId	400	OKlulus
C11	DELETE proyek ke-2	200, sisa 2 proyek	OKlulus
C12	DELETE proyek yang sama lagi	404 proyek	OKlulus
C13	PUT portofolio S2 (3 proyek + 1 sertifikat)	200	OKlulus
C14	GET `/portfolios/stats/technologies`	10 teknologi teratas	OKlulus
—	16 POST proyek beruntun	16×201 → 18 proyek	OKlulus
C15	10 POST proyek **serentak** (sisa 2)	tepat 2×201, 8×409	OKlulus
C16	POST ke-21	409 PROJECT_LIMIT	OKlulus
C17	Stats akhir (18 proyek python/flask ikut terhitung)	cocok hitungan manual	OKlulus
—	explain find per `student_id`	index unik terpakai	OKlulus

3.3 Kasus gagal (sesuai ketentuan: minimal tiga kasus gagal)

C1 — GET portofolio yang belum dibuat (404 portofolio):

```
### C1 | rev001 GET portofolio mahasiswa 70 -> harap 404 'portofolio tidak ditemukan
{"success":false,"code":"NOT_FOUND","message":"portofolio tidak ditemukan","request_
[status 404]
```

C2 — mahasiswa tidak ada di PostgreSQL (404 mahasiswa): membuktikan pemeriksaan lintas database bekerja — request ditolak oleh PostgreSQL sebelum MongoDB disentuh.

```
### C2 | GET portofolio mahasiswa 99999 -> harap 404 'mahasiswa tidak ditemukan'
{"success":false,"code":"NOT_FOUND","message":"mahasiswa tidak ditemukan","request_i
[status 404]
```

C3 dan C4 — bukan pemilik (403): rev002 mencoba menulis portofolio milik rev001, dan rev003 (user biasa tanpa permission any) mencoba membacanya:

```
### C3 | rev002 PUT portofolio milik rev001 -> harap 403
{"success":false,"code":"FORBIDDEN","message":"tidak berhak mengakses portofolio mah
[status 403]

### C4 | rev003 (user biasa, bukan pemilik) GET portofolio 70 -> harap 403
{"success":false,"code":"FORBIDDEN","message":"tidak berhak mengakses portofolio mah
[status 403]
```

C10 — id proyek tidak valid (400): `bson.ObjectIDFromHex` menolak id sebelum query dibuat:

```
### C10 | DELETE proyek dengan id tidak valid -> harap 400 'id proyek tidak valid'
{"success":false,"code":"BAD_REQUEST","message":"id proyek tidak valid","request_id"
[status 400]
```

C12 — DELETE proyek yang sudah terhapus (404, pola whyNoMatch):

```
### C12 | DELETE proyek yang sama lagi -> harap 404 'proyek tidak ditemukan'
{"success":false,"code":"NOT_FOUND","message":"proyek tidak ditemukan di dalam porto
[status 404]
```

3.4 Normalisasi teknologi dan konsistensi `created_at`

```
### C5 | rev001 PUT portofolio lengkap (2 proyek + 1 sertifikat, teknologi kacau) ->
{'proyek': [('API Perpustakaan', ['go', 'postgresql', 'docker']), ('Dashboard Statis

### C7 | PUT ulang (ganti headline) -> harap 200, created_at TETAP
{'created_at': '2026-10-28T08:56:35.775Z', 'updated_at': '2026-10-28T08:56:35.868Z',
```

3.5 NoSQL injection

Body PUT berisi operator MongoDB `{"headline": {"$ne": ""}}`. Body parser + validator menolaknya sebelum menyentuh database (tipe `headline` bukan string):

```
### C9 | NoSQL injection di body PUT: {"headline": {"$ne": ""}} -> harap 400
{"success":false,"code":"BAD_REQUEST","message":"body harus berupa JSON yang valid",
[status 400]
```

Tidak ada data yang berubah — operasi gagal di lapisan validasi, bukan di database.

3.6 Race condition: batas 20 proyek dijaga di level database

Portofolio S1 diisi 16 proyek beruntun (semua 201 → total 18), lalu **10 request POST dikirim serentak** saat sisa tepat 2 tempat:

```
=== RACE: isi S1 sampai 18 proyek (satu request = satu POST) ===
201 201 201 201 201 201 201 201 201 201 201 201 201 201 201 201
jumlah proyek sekarang (harap 18):
18

### C15 | 10 POST proyek serentak saat sisa 2 -> harap TEPAT 2x201, 8x409
201 201 409 409 409 409 409 409 409 409
jumlah proyek akhir (harap 20):
20

### C16 | POST ke-21 -> harap 409 PROJECT_LIMIT
{"success":false,"code":"PROJECT_LIMIT","message":"jumlah proyek sudah mencapai bata
[status 409]
```

Hasilnya **tepat 2 yang berhasil dan 8 ditolak 409** — walaupun 10 request sama-sama lolos validasi, filter atomik `projects.19 $exists false` membuat hanya 2 yang benar-benar menemukan tempat kosong. Batas 20 terjaga di level database, bukan di lapisan aplikasi.

3.7 Statistik teknologi (aggregation `$unwind` ganda)

```
### C14 | GET /portfolios/stats/technologies -> 10 teknologi terpopuler
[{'technology': 'docker', 'total': 3}, {'technology': 'go', 'total': 3}, {'technolog

### C17 | stats akhir (termasuk 18 proyek python/flask)
[{'technology': 'python', 'total': 20}, {'technology': 'flask', 'total': 18}, {'tech
```

Angka C17 cocok dengan hitungan manual: 16 proyek uji race (python+flask) + proyek lain → python 20, flask 18, docker 3, go 3, postgresql 2, django 1, redis 1.

3.8 Bukti index: `explain("executionStats")`

```
=== BUKTI INDEX (mongosh, MongoDB 8.0.30) ===
db.student_portfolios.find({student_id: 70}).explain("executionStats")
  index terpakai  : student_portfolio_unique (EXPRESS_IXSCAN)
  totalDocsExamined : 1
  nReturned : 1
  exec time (ms) : 0
aggregation $unwind ganda: COLLSCAN — sesuai desain, stats membaca SELURUH collectio
  (tanpa $match, index selective apa pun tidak akan dipakai; komentar EnsureIndexes)
```

`totalDocsExamined = 1 = nReturned`: query per mahasiswa menyentuh tepat satu dokumen lewat index unik. Aggregation stats memang COLLSCAN — itu keputusan desain, bukan kelalaian (lihat C.1.3).

3.9 Screenshot MongoDB Compass

Screenshot MongoDB Compass — diisi manual

Tampilan satu dokumen pada koleksi `student_portfolios`
database `praktikum_backend`
(misal dokumen `student_id: 71` pada bagian C.1.2)

C.4 — Jawaban Pertanyaan Analisis

10. Mengapa projects disematkan (embedded) di dalam dokumen portofolio, bukan disimpan sebagai koleksi terpisah? Kapan keputusan ini harus diubah?

Projects disematkan karena tiga syarat embedding terpenuhi sekaligus. Pertama, **pola baca**: tidak ada endpoint yang membaca satu proyek secara sendirian — GET portofolio selalu mengembalikan induk beserta seluruh proyeknya, sehingga satu `Find0ne` menghasilkan dokumen lengkap tanpa join manual. Kedua, **ukuran**: jumlah proyek dibatasi 20 (sertifikat 50), sehingga ukuran dokumen jauh dari batas 16 MB; tumbuhnya terkendali oleh database sendiri lewat filter atomik `projects.19 $exists false`. Ketiga, **tak dipakai lintas dokumen**: tidak ada query "semua proyek ber-isu X di seluruh mahasiswa"; analisis lintas dokumen yang dibutuhkan (statistik teknologi) tetap bisa dijawab dengan `$unwind` atas array embedded. Keputusan ini harus dibalik bila proyek mulai punya **kehidupan sendiri** — misalnya komentar, tanda suka, atau riwayat revisi dari pengguna lain yang dibutuhkan tanpa memuat portofolio induk, atau bila jumlahnya tumbuh tak terbatas — karena array besar yang sering diubah di tengah dokumen yang sekaligus menangani PUT penuh akan membuat update saling menabrak; saat itu projects layak menjadi koleksi terpisah dengan referensi `student_id`.

11. Apa yang terjadi pada portofolio bila mahasiswanya dihapus dari PostgreSQL? Bagaimana menanganinya?

Portofolionya menjadi **data yatim**: dokumen tetap ada di MongoDB, tetapi tak lagi punya induk yang sah — GET portofolionya kini mengembalikan 404 "mahasiswa tidak ditemukan" (karena `student0wned` menolak lebih dulu), data menumpuk tanpa bisa diakses, dan ironisnya **tetap ikut terhitung** dalam statistik teknologi karena pipeline `$unwind` tidak difilter terhadap keberadaan mahasiswa. Dua cara menanganinya: **(a) membersihkan saat penghapusan** — service delete student ikut menghapus `student_portfolios` dengan `student_id` yang sama, atau bila menginginkan jaminan akhir, job berkala mencari dokumen yatim; kelemahannya, tidak ada transaction lintas database, jadi selalu ada celah kecil dokumen yatim antara dua commit, dan proses pembersihannya pekerjaan tambahan yang bisa gagal sendiri. **(b) menghindari penghapusan keras** — mahasiswa nonaktif ditandai `is_active = false` dan datanya dipertahankan, sehingga portofolio tidak pernah yatim; kelemahannya, data menumpuk tanpa batas dan statistik bisa mencantumkan teknologi milik mahasiswa nonaktif kecuali pipeline difilter/dipisahkan. Untuk desain ini pilihan (a) lebih cocok: portofolio adalah milik pribadi mahasiswa yang maknanya hilang bersama pemiliknya, dan endpoint yang ada memang tidak membutuhkan jejak mahasiswa terhapus.

12. Endpoint mana yang tetap hidup dan mana yang gagal bila MongoDB mati? Apakah perilaku ini diinginkan?

Semua endpoint yang datanya di PostgreSQL tetap hidup: autentikasi dan registrasi, manajemen users/students/courses (termasuk KRS), dan seluruh endpoint supervisors — mereka tidak menyentuh MongoDB sama sekali. Yang gagal adalah seluruh fitur berbasis MongoDB: keempat endpoint portofolio (GET/PUT/POST/DELETE proyek) dan statistik teknologi, serta endpoint ringkasan ulasan — semuanya berakhir 500 setelah server-selection timeout karena collection-nya memang tidak ada di tempat lain. Sementara itu `/health` berubah menjadi 503 dan menyebut komponen MongoDB secara eksplisit, sehingga pemantau tahu persis mana yang bermasalah. Perilaku ini **memang yang**

diinginkan untuk arsitektur polyglot persistence: fitur yang datanya di MongoDB mustahil dilayani tanpa MongoDB, tetapi satu komponen yang mati tidak perlu menjatuhkan seluruh aplikasi; isolasi seperti ini justru salah satu alasan data portofolio dan ulasan dipisahkan dari data transaksional mahasiswa sejak awal.